

Stateless vs. Stateful Architecture

Outline

- 1 Introduction & Definitions
- 2 Request Handling & Resource Consumption
- 3 Failure Handling & Security
- 4 Key Use Cases & Pros/Cons
- 5 Hybrid Approaches
- 6 Conclusion

Introduction

- For computing and data processing associated with apps, architectures can be described as either **stateless** or **stateful**.
- **Key difference** is in **how they manage "state"** (**information about previous interactions**).
- **Both models have distinct use cases, strengths, and challenges.**
- Understanding the key differences between these two architectural models
 - is crucial in system design,
 - especially for web services, distributed systems, & cloud computing.

Stateless Architecture

- **Definition:**

- In a stateless architecture, **each request from a client to the server is independent.**
- The **server does not retain any memory of previous interactions**, and each request is processed as if it is being made for the first time.

- **Example:** **HTTP is a stateless protocol.**

- Every HTTP request is treated independently, meaning the server does not store any session information between requests unless explicitly provided (e.g., via cookies or tokens).

Stateful Architecture

- **Definition:**

- In a stateful architecture, the server retains session information or state across multiple requests from the same client.
- The state allows the server to remember previous interactions and provide context-aware responses.

- **Example:** A database connection that remains open and retains state across multiple queries is a stateful system.

- Likewise, traditional FTP sessions, where the connection maintains state and can be used for subsequent data transfers, is also stateful.

Architecture Models: Stateless vs Stateful

The Foundational Choice: Memory & State

Stateless Architecture

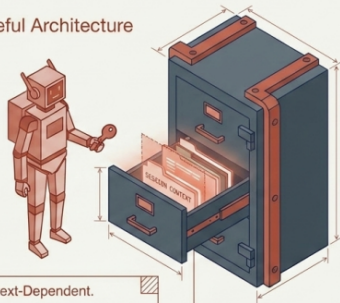


Independent Request.

The server retains zero memory of previous interactions. Every request contains all required information (auth, state, data) to process it from scratch.

[e.g., HTTP, REST APIs]

Stateful Architecture



Context-Dependent.

The server establishes and retains session information across multiple requests. The client only needs to pass an identifier to unlock the ongoing state.

[e.g., Database Connections, FTP]

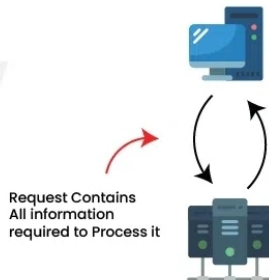
Architecture Models: Stateless vs Stateful ...cont'd

Stateful Architecture



VS

Stateless Architecture



Request Handling: Stateless

- **Request Independence:**

Every request is self-contained, meaning all information required to process the request must be included in the request itself.

Note: The server does not rely on past interactions.

- **Data Transmission:**

Since no session data is stored on the server, the client must send all relevant data with each request (e.g., authentication tokens, request parameters).

- **Example:**

In a stateless REST API, each request must include necessary data like authentication tokens because the server does not keep track of any session or user state.

Request Handling: Stateful

- **Request Dependency:**

Each request is processed with context from previous interactions.

Note: Server retains session data or other state info. between requests.

- **Data Transmission:**

- The client does not need to send all information with each request.
- Once the state is established, the server uses that state to process further requests efficiently.

- **Example:**

In a stateful online banking application, once a user logs in, the system retains their session, allowing the user to navigate different pages without needing to re-authenticate or resend information.

Resource Consumption: Stateless

- **Lower Server Memory Usage:**

Since the server doesn't need to store client session information or state, it requires fewer resources (e.g., memory).

- **Scalability:**

Stateless systems are typically more scalable because,

- each request is treated independently, making it easier to distribute load across multiple servers or instances in a load-balanced system.

- **Example:**

A stateless microservices architecture can be easily scaled by adding more instances without worrying about session management.

Resource Consumption: Stateful

- **Higher Server Memory Usage:**

Maintaining state requires additional resources on the server, including memory and storage, as each client session is stored.

- **Scalability Challenges:**

- Stateful systems are harder to scale because they require session data to be shared across servers or maintained on the same server.
- Scaling stateful applications often involves strategies like session persistence or sticky sessions, which can complicate architecture.

- **Example:**

A stateful multiplayer online game server must maintain player states (position, health, inventory), which requires careful memory and resource management.

Failure Handling: Stateless

- **Resilience to Failure:**

- In stateless systems, failure of a server or service has minimal impact.
- Since no session information is stored, another server can handle the next request without issues.

- **Easier Recovery:**

If a stateless server crashes, the system can simply redirect incoming requests to another instance without needing to recover the session.

- **Example:**

In a stateless web service, a load balancer can route traffic to any available server instance, even if one fails.

Failure Handling: Stateful

- **Complex Failure Recovery:**

- If a stateful server fails, the session data could be lost unless the state is replicated or shared across other servers.
- This requires strategies like session persistence, replication, or using databases to store session data.

- **Server-Specific Dependencies:**

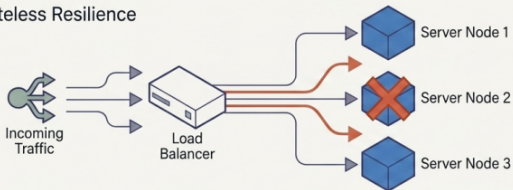
Stateful architectures often create dependencies on specific servers, making it harder to switch to another server in the event of failure.

- **Example:**

In a stateful database server, if the server crashes without replication, transactions and session data might be lost.

System Implications: Scaling and Failure

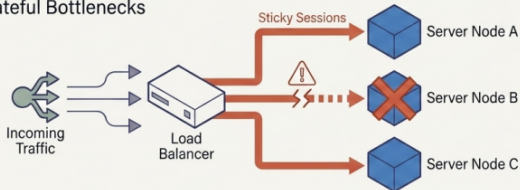
Stateless Resilience



With no session data stored locally, any server can handle any request. Failure means instant, lossless rerouting.

[e.g., Auto-Scaling, Instant Failover]

Stateful Bottlenecks



Because a specific server holds the user's active session state, node failure results in dropped sessions unless complex database replication is implemented.

[e.g., Session Loss, Complex Replication]

- **Client-Side Security:**

- Stateless systems often rely on clients to store session information (e.g., using tokens or cookies), which shifts some of the security responsibilities to the client.
- Care must be taken to secure the transmission and storage of tokens.

- **Token-Based Authentication:**

- Stateless systems frequently use token-based mechanisms such as JWT (JSON Web Tokens), which include all necessary session data in the token itself.
- Care must be taken to protect these tokens from being hijacked.

- **Example:**

A stateless API might use JWTs, where the token contains the session information and is sent with each request.

- **Server-Side Security:**

- Since the server retains session information, the responsibility for protecting this data rests with the server.
- This can be both an advantage (centralized control) and a risk (if compromised).

- **More Vulnerable to Attacks:**

Storing session data on server opens the door to attacks like session hijacking or session fixation if security is not properly managed.

- **Example:**

A stateful web application that maintains a session ID for each user may be vulnerable to session hijacking if the session ID is exposed.

- **Best for Simple, Idempotent Requests:**

- Stateless architecture is ideal for use cases where each request can be handled independently, without relying on the previous context.
- This includes APIs, microservices, and RESTful services.

- **Cloud-Based Architectures:**

In cloud-based services, stateless designs enable easy horizontal scaling, as instances can be added or removed without session management issues.

- **Examples:**

- REST APIs
- Microservices
- Serverless architectures (e.g., AWS Lambda, Google Cloud Functions)

Use Cases: Stateful

- **Best for Complex, Multi-Step Interactions:**

Stateful architectures are needed when the application must remember previous interactions to complete complex processes (e.g., user sessions in web/mobile apps).

- **Real-Time Systems:**

Systems like online gaming, chat services, or applications requiring real-time data management often rely on stateful architecture to track user progress or interactions.

- **Examples:**

- Online banking systems
- E-commerce websites with shopping carts
- Multiplayer online games
- Persistent database connections

Pros and Cons: Stateless

Pros:

- Easy to scale and distribute across multiple servers.
- Easier fault tolerance and recovery.
- Reduced memory and resource usage on the server.
- Simpler to develop for services that do not require maintaining user state.

Cons:

- Every request must include all necessary data (e.g., tokens).
- Less efficient for applications where context must be retained across multiple requests.
- Shifts some security and state management to the client.

Pros and Cons: Stateful

Pros:

- Efficient for managing multi-step processes or sessions.
- Suitable for complex applications that require context or user-specific data.
- Centralized control of session data can simplify certain security measures.

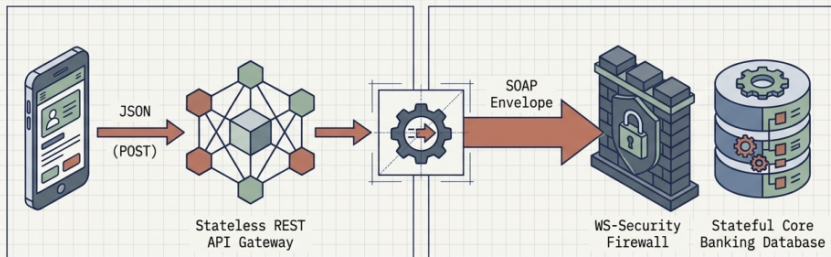
Cons:

- Harder to scale due to the need for session persistence.
- More vulnerable to certain types of attacks (e.g., session hijacking).
- Increased memory and resource usage on the server to maintain state.

Hybrid Approaches

- Many modern apps combine stateless and stateful components.
- For instance, **stateless front-end services** can communicate with **stateful back-end services** where session management is needed.
- **Example:**
In a web application,
 - the front-end may use stateless REST APIs for basic data retrieval,
 - while the back-end maintains stateful database connections and session management for user authentication and complex transactions.

The Hybrid Reality of Modern Systems



The Edge: REST handles massive, unpredictable front-end traffic with lightweight horizontal scaling.

The Core: The microservice passes the transaction deep into the legacy backend, utilizing SOAP for ACID-compliant, secure stateful integrity.

Modern architecture relies on REST for scalable front-end distribution, and SOAP for secure backend transactional integrity.

Conclusion

- **Stateless architecture:** Ideal for scalable, simple systems with independent requests.
- **Stateful architecture:** Necessary for applications requiring complex interactions and state management.
- Many systems today use a **hybrid approach** to leverage the advantages of both architectures.

Thank you